

Backend Architecture & System Design — agriQ Harish Facility

The system follows an event-driven model with four layers: ingestion, processing, storage, and delivery. Below is a comprehensive breakdown of the architecture, database structure, risk logic, alerting, and ERP integration powering the agriQ Harish facility.

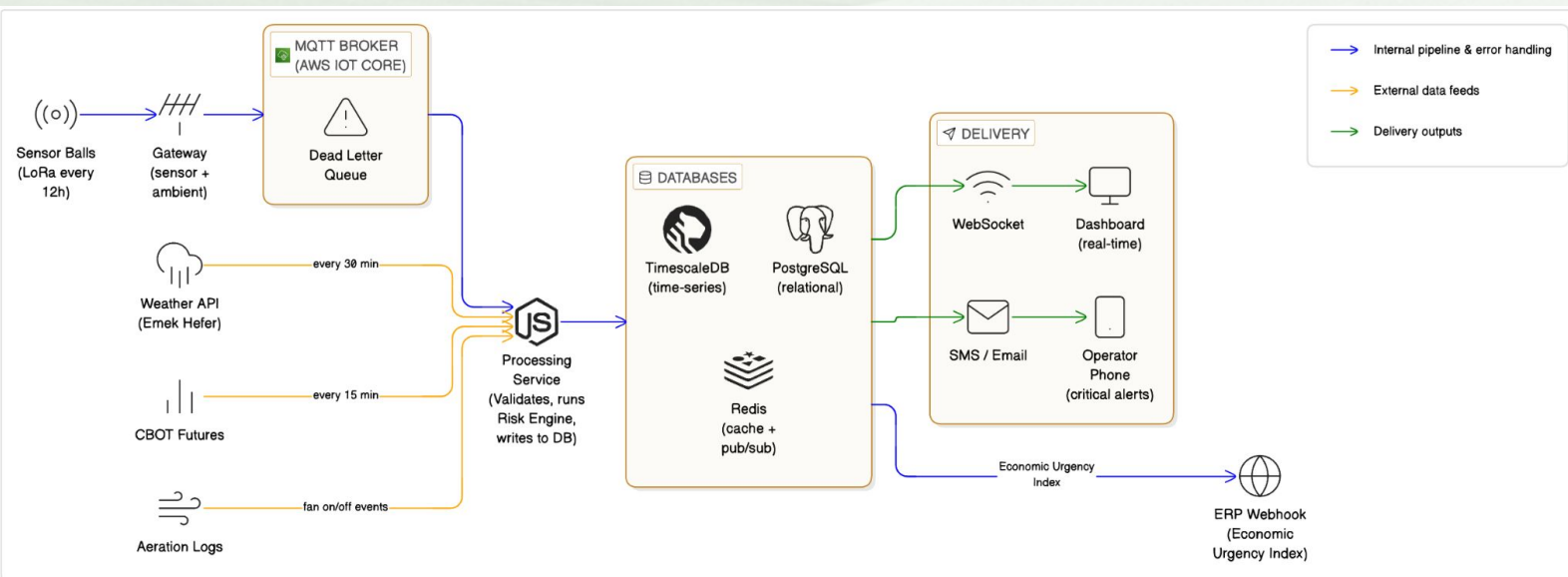
1. Architecture & Data Flow

Sensors operate in Deep Sleep mode - they wake every 12 hours, push their payload (`sensor_id`, `timestamp`, `temperature`, `moisture`, `battery_voltage`) to the facility Gateway via LoRa, and sleep immediately. This extends battery life from weeks to 12+ months. Trade-off: sensors cannot be pinged on demand.

The Gateway publishes all readings (both sensor balls and its own ambient temp/humidity) to a Message Broker (AWS IoT Core via MQTT). This decouples ingestion from processing - if the backend is down during a transmission window, messages queue safely. Consumers are idempotent: duplicates are discarded via a (`sensor_id`, `timestamp`) compound key. A dead-letter queue catches failures after 3 retries.

A Node.js processing service consumes from the broker, validates readings, runs the Risk Engine, and writes to the databases. It is stateless and horizontally scalable. Separately, scheduled jobs (BullMQ) poll the Emek Hefer Weather API every 30 minutes and CBOT wheat futures every 15 minutes during market hours.

Additional data source — Aeration System Logs: Temperature drops cannot be understood without knowing if the operator turned on the fans. Integrating aeration events (fan on/off, speed) prevents false diagnostics and enables future automated cooling.



The architecture cleanly separates concerns: external feeds (weather, futures, aeration) enrich the processing layer, while the database tier fans out to both real-time dashboard delivery and ERP webhook outputs — ensuring no single point of failure disrupts the full pipeline.

2. Database Structure

Polyglot persistence — TimescaleDB for time-series, PostgreSQL for relational data, Redis for caching and pub/sub.

TimescaleDB Hypertables

`sensor_readings` (`time`, `sensor_id`, `pile_id`, `temperature`, `moisture`, `battery_pct`)
`gateway_readings` (`time`, `gateway_id`, `pile_id`, `ambient_temp`, `ambient_humidity`)
`weather_data` (`time`, `external_temp`, `humidity`, `wind`, `conditions`)
`aeration_events` (`time`, `pile_id`, `event_type`, `fan_speed_pct`)

Redis

Caching: Latest reading per sensor (key: `latest:{pile_id}:{sensor_id}`) for instant dashboard loads.
Pub/Sub: Processing service publishes alert events, WebSocket servers relay to connected browsers.

Retention Policy

Raw data kept 2 years, then downsampled to daily averages. Alert history kept indefinitely for audits.

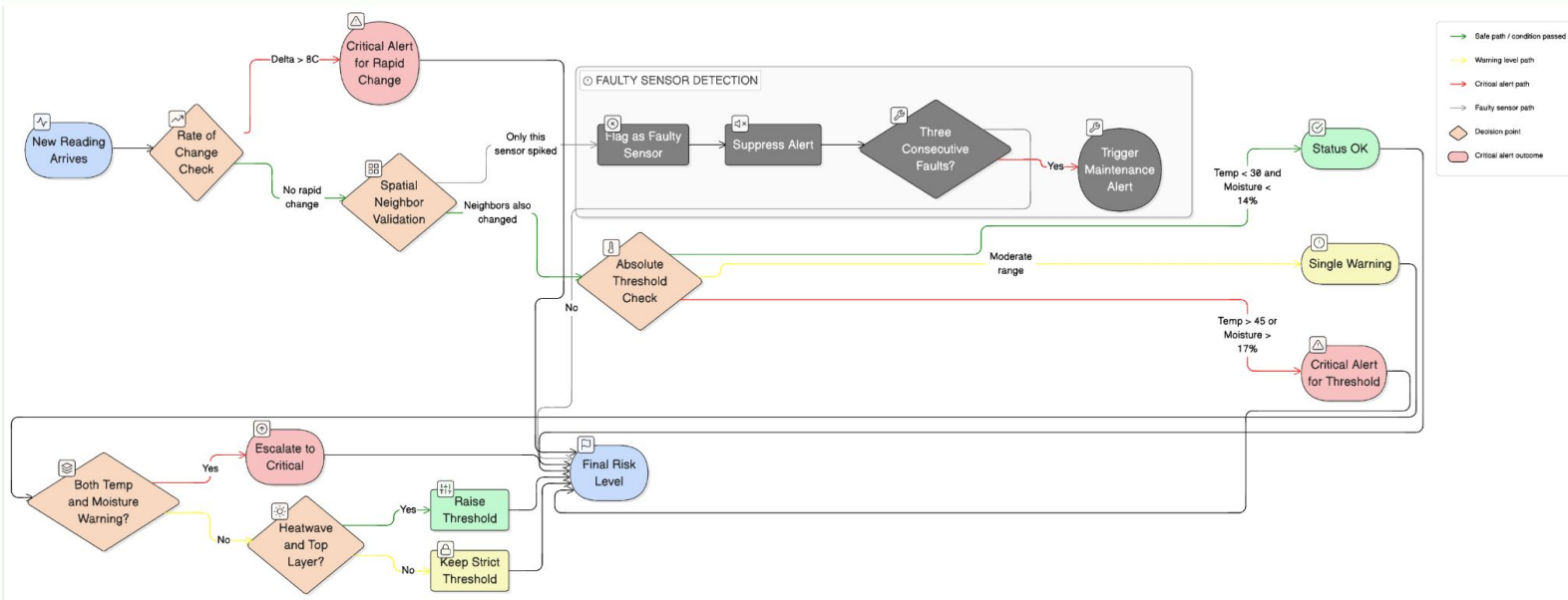
PostgreSQL Tables

`sites` (`id`, `name`, `address`, `lat`, `lng`)
`piles` (`id`, `site_id`, `name`, `dimensions`, `grain_type`, `tonnes`)
`sensors` (`id`, `pile_id`, `layer`, `position_x/y/z`, `status`, `installed_at`)
`alerts` (`id`, `pile_id`, `sensor_ids[]`, `severity`, `created_at`, `resolved_at`, `action_taken`)
`users` (`id`, `name`, `role`, `email`, `phone`, `notification_prefs` JSONB)
`commodity_prices` (`id`, `price_usd`, `fetches_at`)

Risk Logic, Alerting & ERP Integration

3. Risk Logic

Risk is not a single threshold — it is the interaction of multiple variables across time and space.



Rate-of-change is checked before absolute thresholds for proactive alerting. The flowchart above captures the full decision sequence — from raw reading to final risk classification — including faulty sensor suppression and dynamic weather-based threshold adjustment.

Faulty Sensor Detection
Uses 3D spatial interpolation (inverse-distance-weighted neighbors) to validate. Three consecutive faulty cycles trigger maintenance alert.

Multiplier Effect
Both temp and moisture in Warning range = escalate to Critical immediately.

Rate of Change
Gradual (0.5–1°C/cycle) → Watch state; 3+ cycles escalates. Sudden (>8°C/cycle) → skip Warning, immediate Critical + SMS.

Weather Context
External >38°C for 2+ readings → top-layer warning shifts 30→35°C. Middle/bottom always strict.

Sensor Health
Two missed transmissions → maintenance alert. Battery <20% → warning, <10% → replace.

4. Alerting & Notifications

WebSockets push all state changes to the dashboard in real time. For off-screen operators: Critical alerts trigger SMS via Twilio (example: "CRITICAL: Emek East, Middle Layer — 51°C / 18.4% moisture. Immediate aeration required."). Operators configure preferences per channel in their profile. Daily email digests go to managers.

Authentication uses JWT via Auth0, with role-based access:

Operator
View + acknowledge alerts

Manager
+ Configure thresholds, view ERP recommendations

Admin
+ System configuration

5. ERP & Commodity Integration

An Economic Urgency Index combines deterioration rate with market conditions:

$$\text{Urgency} = \frac{\Delta\text{Temp} + \Delta\text{Moisture}}{\text{Time_to_Spoilage}} \times \frac{\text{CBOT_Price}}{\text{Historical_Avg}}$$

When the index exceeds a threshold, the system sends a webhook to the ERP with pile status, projected spoilage timeline, and recommended action (e.g., "Sell Emek South now — prices peaked, quality declining").

Key Trade-offs

Decision	Chosen Approach	Trade-off
Deep Sleep over always-on	Battery life 12+ months	No on-demand pinging
Polyglot DB over single PostgreSQL	Query performance optimized per data type	Operational complexity
WebSockets over SSE	Bidirectional for future alert acknowledgment	More complex than read-only SSE
Dynamic top-layer thresholds over static	Reduces false alerts during heatwaves	Added complexity